

TECHNICAL DOCUMENTATION & SYSTEM DESIGN

SHERLOCK

AI

Intelligent Maintenance Wizard for Steel Manufacturing

ABOUT THIS DOCUMENT

A complete engineering reference: architecture, technology stack, data & reasoning pipeline, alerting and prediction logic, deployment, and sample I/O.

VERSION

1.0 — Engineering Edition

STACK

Next.js · FastAPI · LangGraph

AI

Gemini 2.5 Flash

AUTONOMOUS PLANT INTELLIGENCE SYSTEM

System at a Glance

Sherlock AI consolidates operational data, maintenance records, live sensor streams and engineer queries into one decision-support platform — explainable diagnosis, failure prediction, and prioritized action.



END-TO-END PIPELINE



DESIGN PRINCIPLE

Every recommendation is traceable to the sensor evidence that produced it — auditable intelligence, never a black box.

System Architecture

PRESENTATION TIER

Next.js · React · TypeScript · Tailwind · Three.js · Recharts – dashboard, 3D twin, AI chat, reports



SERVICE TIER · REST API

FastAPI · Uvicorn · JWT · CORS · GZip – /auth /dashboard /maintenance /safety /energy /production /chat /reports /demo



INTELLIGENCE TIER

LangGraph StateGraph · supervisor → 5 domain agents · Gemini 2.5 Flash (OpenRouter) · rule-based fallback



REAL-TIME ENGINE

live_sensor background thread · 10s cadence · failure-probability model · threshold-breach alerting



DATA LAYER

data_service over CSV (plant · maintenance · energy · safety) via pandas · reportlab reports · future RAG KB

Technology Stack

FRONTEND

- Next.js 14
- React 18
- TypeScript
- Tailwind CSS
- Three.js
- Recharts
- Axios
- Lucide React
- Date-fns

BACKEND

- FastAPI
- Uvicorn
- Python 3.11+
- JWT Auth
- Passlib + Bcrypt
- Pydantic Settings
- Dotenv
- CORS Middleware
- GZip

AI & MACHINE LEARNING

- LangGraph
- Gemini 2.5 Flash
- OpenRouter API
- Multi-Agent AI
- Explainable AI
- Failure Prediction

DATA PROCESSING

- Pandas
- NumPy
- CSV Data Sources

REPORTING

- ReportLab
- PyPDF

DATABASE (READY)

- PostgreSQL
- SQLAlchemy
- AsyncPG

DEVOPS & DEPLOYMENT

[Docker](#)[Docker Compose](#)[Vercel - Frontend](#)[Render - Backend](#)

Data & System Flow

DASHBOARD LOAD

- 1 User authenticates; token attached to calls
- 2 Page calls `/api/dashboard/kpis + timeseries`
- 3 Routes read `data_service + live_sensor`
- 4 GZip JSON returned; UI renders charts & twin
- 5 Live endpoint polled — alerts update live

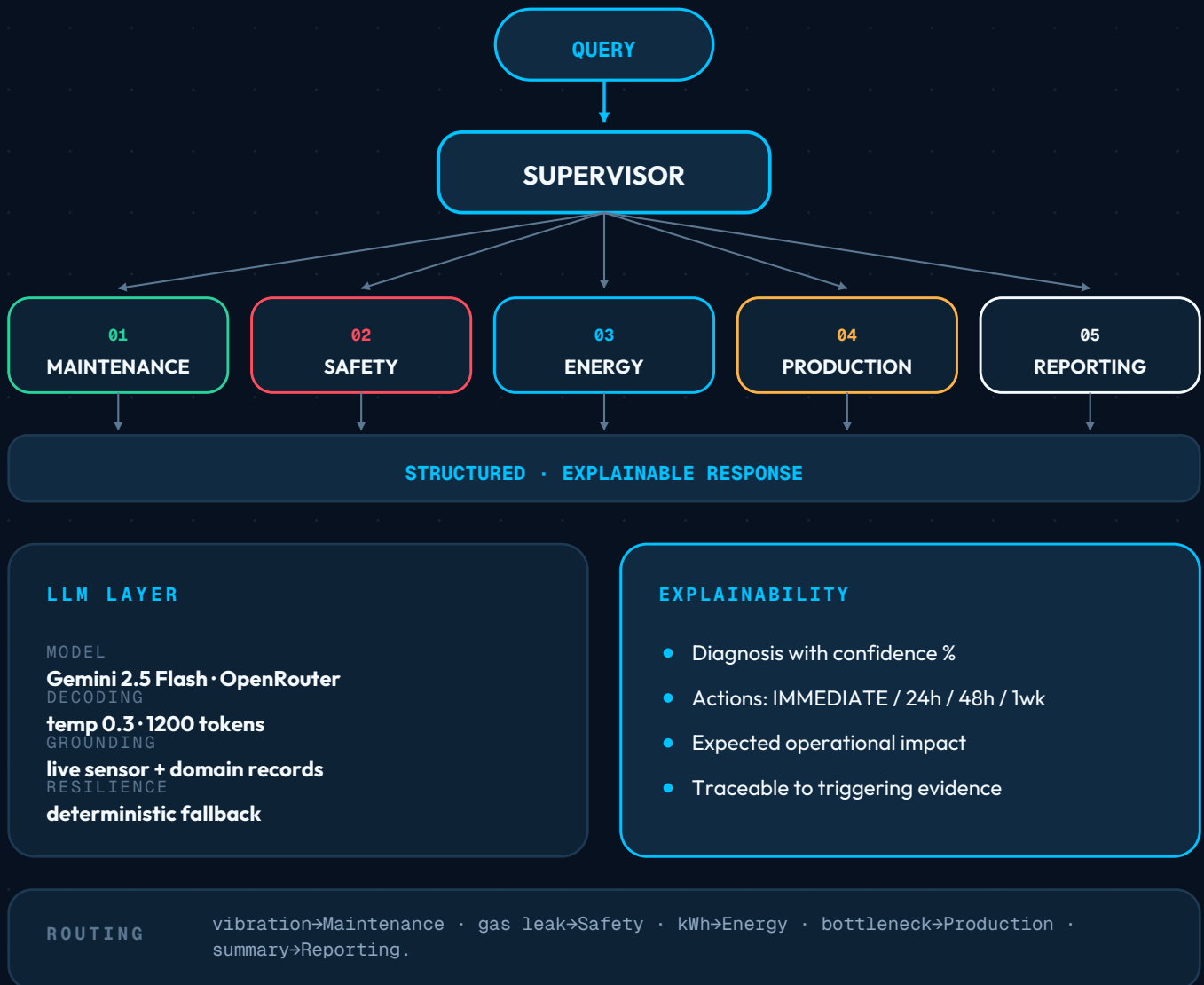
AI QUERY

- 1 Engineer posts NL query to `/api/chat`
- 2 Supervisor classifies intent & routes
- 3 Agent builds context-grounded prompt
- 4 Gemini returns structured answer
- 5 Rendered; follow-ups retain context

TWO STREAMS

Historical CSV knowledge + a live 10-second condition-monitoring stream feed one unified API contract.

Model Design & Reasoning



Alerting & Prediction Logic

FAILURE-PROBABILITY MODEL

$$fp = 0.35 \cdot vib + 0.40 \cdot temp + 0.25 \cdot pres$$

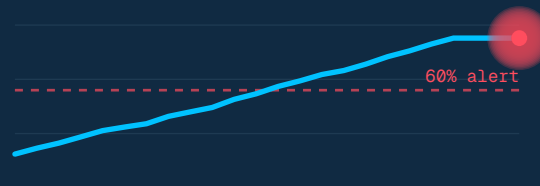
+ noise → clamped to [0.01, 0.99] · recomputed every 10 seconds

**NORMAL** **$fp \leq 0.50$** **WARNING** **$0.50 - 0.75$** **CRITICAL** **$fp > 0.75$**

THRESHOLD-BREACH EVENT

```
{
  "type": "THRESHOLD-BREACH",
  "unit": "Blast-Furnace-1",
  "severity": "CRITICAL",
  "value": 0.78,
  "is_live": true
}
```

FAILURE PROBABILITY



DETECTION LOGIC

- A background thread random-walks vibration, temperature, pressure & production every 10s.
- Crossing 60% failure probability emits a HIGH (or CRITICAL >75%) live event into a rolling buffer.
- Demo injection spikes a unit's sensors to showcase reactive detection end-to-end.

Assumptions & Limitations

Simulated telemetry

Live values come from a realistic random-walk simulator; the data contract is built to swap in a real OT/IoT feed (MQTT/OPC-UA).

Demo datasets

Historical records are representative CSV samples; some dashboard figures are illustrative for the demo.

Knowledge base

Reasons over operational data + LLM priors today; a RAG layer over manuals & SOPs is designed-for and planned next.

Feedback loop

Engineer confirm/correct feedback is on the roadmap; recommendations are currently generated per request.

LLM dependence

Online reasoning needs an OpenRouter key; without it, deterministic rule-based responses keep the system live.

Installation, Configuration & Run

Prerequisites: Python 3.11+ · Node.js 18+ · (optional) OpenRouter API key

BACKEND

```
$ cd backend
$ python -m venv .venv
$ .venv\Scripts\activate
$ pip install -r requirements.txt

# backend/.env (do NOT commit)
# OPENROUTER_API_KEY=...
# ALLOWED_ORIGINS=["http://localhost:3000"]

$ uvicorn app.main:app --reload --port 8000
# docs -> http://localhost:8000/api/docs
```

FRONTEND

```
$ cd frontend
$ npm install

# frontend/.env.local
# NEXT_PUBLIC_API_URL=http://localhost:8000

$ npm run dev
# -> http://localhost:3000

# production:
$ npm run build && npm start
```

ONE-COMMAND (DOCKER)

```
$ docker-compose up --build # backend + frontend together
```

Sample Input & Output

A — Natural-language maintenance query

INPUT (AI COMMAND CHAT)

> "Why is Blast Furnace-1 showing high vibration and what should I do?"

OUTPUT — STRUCTURED & EXPLAINABLE

Elevated vibration (7.4 m/s²) and temperature (1,435 C) — consistent with advancing bearing wear.

Recommended actions (prioritised):

1. [IMMEDIATE] Reduce BF-1 load 15%, inspect bearing within 6h.
2. [24 h] Replace primary thrust bearing (~8h outage, INR 4.2L).
3. [48 h] Vibration audit of Sinter Plant conveyor.

Confidence: 87% | Prevents ~72h downtime, saves INR 18.5L.

B — Injected demo alert (the on-stage 'Simulate' control)

INPUT / EFFECT

```
POST /api/demo/inject { plant: 'Plant-D', type: 'CRITICAL_FAILURE' }  
-> spikes the unit's sensors (vibration 8.9, temp 1580, fp 0.91)  
-> pushes a CRITICAL event to the live feed  
-> dashboard + digital twin recolor that asset to red in one poll cycle
```

Mapped to the Brief

REQUIREMENT	DELIVERED IN SHERLOCK AI
✓ Probable fault diagnosis	AI Insights + maintenance agent
✓ Root-cause analysis	Structured diagnosis + evidence
✓ RUL prediction	Failure-prob trajectory · twin RUL mode
✓ Early failure warning	Threshold-breach live events
✓ Risk classification	NORMAL / WARNING / CRITICAL bands
✓ Action prioritization	Queue by urgency · spares · lead time
✓ LLM contextual reasoning	Gemini 2.5 via LangGraph agents
✓ NL multi-turn interaction	Sherlock AI Command chat
✓ Explainable recommendations	Confidence · actions · impact · trace
✓ Real-time alerting	Live alarm feed + KPI deltas
✓ Reporting outputs	Executive reports · reportlab PDF
✓ IoT / monitoring dashboard	Live monitoring + 3D digital twin

Every core requirement of the problem statement maps to a shipped capability.

Engineered to be explainable.

Real-time, evidence-linked maintenance intelligence for steel-plant reliability testing

AT A GLANCE

3-tier architecture

LangGraph multi-agent

Gemini 2.5 Flash

Live failure model

3D digital twin

Explainable output

RUN

`docker-compose up --build`

API docs at `/api/docs`